

Gradient-Based Structural Estimation

Victor Duarte*

April 20, 2018

Abstract

In this paper I show how gradient-based optimization methods can be used to estimate stochastic dynamic models in economics. By extending the state space to include all model parameters, I show that we need to solve the model only once to do structural estimation. Parameters are then estimated by minimizing the distance between key empirical moments and the model-implied ones. Unlike the Simulated Method of Moments, the model-implied moments are estimated without the computation of a single moment. Instead, a neural network learns the corresponding moments using raw simulated observations. Once a network learned the (differentiable) mapping between parameters and moments, a Newton-Raphson routine is coupled with simulated annealing to find the set of parameters that globally minimizes the objective function. I illustrate the algorithm by solving and estimating a benchmark macroeconomic model with stochastic volatility, endogenous labor supply and irreversible investment.

JEL classification: C15, C61, C63, C68, C45.

Keywords: DSGE, Structural Estimation, Dynamic Programming, SMM.

*MIT Sloan School of Management: vduarte@mit.edu. The paper benefited from comments by Federico Bandi, Diogo Duarte, Julia Fonseca, Leonid Kogan, Jun Li, Jonathan Parker, Adrien Verdelhan and David Thesmar. All errors are my own.

1 Introduction

Structural estimation links rigorous economic models to data, and as such, is a fundamental tool in many fields of economics, from asset pricing and corporate finance to macroeconomics and industrial organization.¹ It is also thought to be a particularly difficult endeavor since it involves solving complicated nested problems. A typical structural estimation procedure entails stochastic dynamic optimization and Monte Carlo approximation for the model-implied moments for a fixed set of parameters in the *inner* loop, and the minimization of a *distance* function of the parameters in the *outer* loop.

This problem is computationally intensive for at least two reasons. Standard approaches to structural estimation solve a dynamic optimization problem and compute simulated moments for each possible configuration of the model parameters. In addition, the optimization algorithm used in the outer loop is oblivious to the mapping between the model parameters and the model-implied moments. In this paper I propose a solution for these two problems.

I leverage new powerful approximate dynamic programming techniques that allows for the global solution of high-dimensional stochastic dynamic problems that previously could be solved only with local approximation methods.² Following [Norets \(2012\)](#), I augment the state space by treating the model parameters as state variables and solve this high-dimensional problem using an extension of the algorithm proposed in [Duarte \(2018\)](#). This makes it unnecessary to solve a complex dynamic programming problem thousands of times.

I then show that one can approximate the model-implied moments without the computation of a single moment. Instead, a deep neural network learns the moments by observing raw outcomes of simulations. This eliminates the need to compute moments

¹See [Durham \(2006\)](#), [Hennessy and Whited \(2005\)](#), [Smith \(2005\)](#) and [Berry et al. \(1995\)](#) for examples.

²See [Scheidegger and Bilonis \(2017\)](#) and [Duarte \(2018\)](#) for examples.

for every fixed set of parameters. More importantly, since the network learns the mapping between parameters and moments (the *binding function* of [Gourieroux et al. \(1993\)](#)), the optimizer in the outer loop can explore this structure to find the model parameters that minimize the corresponding loss function. Specifically, the optimizer has access to analytical gradients and Hessians of the loss function.

The second contribution of the paper is to generalize the algorithm proposed in [Duarte \(2018\)](#) to cases where the optimal controls are defined only implicitly. I do so by parametrizing the policy function using a neural network and updating it with policy gradients. The algorithm doesn't rely on first-order conditions and can accommodate problems with constraints.

The method advanced here fits into the class of moment-based estimators. [Hansen \(1982\)](#) proposed the Generalized Method of Moments (GMM) to estimate deep parameters of stochastic dynamic models from the Euler equations when these can be obtained on closed-form. [McFadden \(1989\)](#), [Pakes and Pollard \(1989\)](#), [Lee and Ingram \(1991\)](#) and [Duffie and Singleton \(1993\)](#) proposed extensions of the conventional methods of moments that can handle generic moment conditions by using simulation. The main difference between these methods and the approach described in this paper is the method used to approximate the model-implied moments. Simulated Method of Moments (SMM) approximate model-implied moments by Monte Carlo simulations for each set of parameters, whereas I learn a mapping between parameters and moments for a whole range of parameters directly from simulated observations. This subtle distinction has big implications in terms of computing time, as the simulation part can be removed from the optimization loop.

The rest of the paper is organized as follows. Section [2](#) extends the value iteration algorithm of [Duarte \(2018\)](#) to do policy iteration, which can handle cases where the optimal controls are defined only implicitly. Section [3](#) shows how a neural network can

learn model-implied moments for a whole range of parameters. Section 4 applies these methods to a benchmark Dynamic Stochastic General Equilibrium (DSGE) model with endogenous labor supply, stochastic volatility and irreversible investment, and section 5 concludes.

2 Approximate Policy Iteration

Compared to models set in discrete time, continuous-time models typically render cleaner first-order conditions, often leading to closed-form solutions for the optimal controls as a function of the value function. When the controls are defined only implicitly, however, the problem becomes substantially more challenging. In that case, value iteration algorithms requires a root finding procedure at each iteration, which could make the iteration so slow as to become unfeasible.³

Many interesting problems in economics, however, involve optimal controls that are defined only implicitly. Examples include models of optimal investment subject to capital constraints, intermediary-based asset pricing and strategic sovereign default. In this section I show how policy iteration can be combined with stochastic gradient descent to solve this class of problems with little additional computational cost.

To fix ideas, the agent's goal is to maximize his lifetime utility subject to some known dynamics:

$$V(\mathbf{X}) = \max_{\mathcal{C}^*} \mathbb{E} \int_0^{\infty} U(\mathbf{C}_t) dt \quad (1)$$

³See Brunnermeier and Sannikov (2016) and Achdou et al. (2014) for examples of value iteration in continuous time.

$$d\mathbf{X} = \mu_X(\mathbf{X}, \mathbf{C})dt + \sigma_X(\mathbf{X}, \mathbf{C})d\mathbf{Z}$$

Where V is the agent's value function, $\mathbf{X}_t$ is a d -dimensional vector of the relevant states, $\mathbf{C}$ is a vector of controls, the drift μ_X is a $d \times 1$ vector, the instantaneous volatility σ_X is $d \times s$ matrix, and $d\mathbf{Z}$ is a $s \times 1$ vector of independent Brownian shocks.

Starting from an arbitrary initial policy function C^0 , policy iteration alternates between the *policy evaluation* step, where the value function V^{C^0} associated with this policy is evaluated, and a *policy improvement step*, where C is updated. In general, these steps can be written as:

$$V^{C^0}(\mathbf{X}) = \mathbb{E} \int_0^\infty U(\mathbf{C}_0((\mathbf{X}_t)))dt \quad (\text{evaluation}) \quad (2)$$

$$C_1 = \arg \max_{C_1} \left\{ U(C_1) - \rho V^{C^0}(\mathbf{X}) + \frac{\mathbb{E} dV^{C^0}}{dt}(\mathbf{X}, \mathbf{C}_1) \right\} \quad (3)$$

$$= \arg \max_{C_1} HJB(V^{C^0}, \mathbf{X}, \mathbf{C}_1) \quad (\text{improvement})$$

The objective of approximate policy iteration is to solve equations 2 and 3 approximately. Specifically, we can parametrize the value and policy functions by neural networks with parameters Θ_V and Θ_C , and iteratively adjust Θ_V and Θ_C so that equation 2 and 3 hold approximately.⁴

The evaluation step can be implemented as in [Duarte \(2018\)](#):

⁴ In the literature of machine learning, where the system dynamics are typically unknown, algorithms that alternate the policy network improvement with the value network prediction are known as *actor-critic* methods. See [Sutton \(1988\)](#), [Szepesvari \(2010\)](#) and [Bertsekas \(2017\)](#) for in-depth treatments of these algorithms in discrete-time settings.

$$\Delta\Theta_V = -\alpha \frac{\sum_{i=1}^n HJB_i \nabla_{\Theta_V} V^C}{n} \quad (4)$$

where α is a learning rate and $\{HJB_i\}_{i=1}^n$ is the HJB residual evaluated at a randomly sampled minibatch of the training set $\{X_i\}_{i=1}^N$. After a few iterations of this prediction step, we can apply the stochastic gradient ascent on equation 3 to improve the policy network⁵:

$$\Delta\Theta_C = \alpha \frac{\sum_{i=1}^n \nabla_{\Theta_C} HJB_i}{n} \quad (5)$$

Implicit in equation 5 is the assumption that $HJB(V^{C_0}, \mathbf{X}, \mathbf{C1})$ is convex in $\mathbf{C1}$, which is typically the case in economic applications.

Policy iteration can easily handle a wide class of constraints on the controls. For example, if labor L is restricted to be in the interval $[0, 1]$, we can impose the parametric form $L(\mathbf{X}) = \sigma(DNN(\mathbf{X}))$, where σ is the logistic function and $DNN(\mathbf{X})$ is a generic network with range $(-\infty, \infty)$. Likewise, if a firm is subject to a borrowing constraint of the form $B \leq \lambda A$, we can set the borrowing policy as $B(\mathbf{X}) = A \cdot \sigma(DNN(\mathbf{X}))$. Notice that these assumptions are without loss of generality, since neural networks can, in principle, approximate any continuous function.

The main advantage of parametrizing the value and policy functions with deep neural networks is their ability to represent high-dimensional functions. Following Norets (2012), I augment the state space by including all model parameters as state variables. More precisely, the value and policy networks take as inputs both state and parameters: $V = V(\mathbf{X}, \boldsymbol{\beta})$ and $C = C(\mathbf{X}, \boldsymbol{\beta})$ ⁶. Once the policy iteration is finished, we have the policies for a whole range of parameters.

⁵See appendix C.1 for an heuristic derivation of equation 5

⁶Notice that update rules 4 and 5 remain unchanged.

3 Learning the moments

In section 2 I showed how to avoid the major bottleneck of moments-based structural estimation: the need to solve a complex dynamic programming problem at each iteration of the outer loop. In this section I show that we can break through the second bottleneck: the need to simulate moments at each iteration of the outer loop.

Let β be the vector of the stacked parameters of a given dynamic model and Y a particular variable of interest. We are interested in learning the function

$$g(\beta) \equiv \mathbb{E}[Y|\beta] \tag{6}$$

Notice that g satisfies⁷:

$$g(\beta) = \arg \min_f \{ \mathbb{E} (Y - f(\beta))^2 \} \tag{7}$$

Problem 8 is a standard *supervised learning* problem, and can be solved with stochastic gradient descent. Here I outline only the main ideas and refer the reader to [Goodfellow et al. \(2016\)](#) for a more detailed discussion.

I will look for a neural network $\hat{g}(\beta; \Theta)$ that approximately solves equation 7, that is:

$$\Theta^* = \arg \min_{\Theta} \{ \mathbb{E} (Y - \hat{g}(\beta; \Theta))^2 \} \tag{8}$$

where Θ are the network parameters (the weights and biases).

Suppose we have a large and i.i.d sample of pairs $\{\beta_i, Y_i\}_{i=1}^N$, that is, we have raw

⁷See appendix ?? for a derivation.

observations of the variable of interest Y for many random draws of the parameter vector β .⁸ Substituting the population mean by the sample mean we get:

$$\Theta^* \approx \arg \min_{\Theta} \mathcal{L}(\Theta) \quad (9)$$

where

$$\mathcal{L}(\Theta) = \frac{\sum_{i=1}^N (Y_i - \hat{g}(\beta_i; \Theta))^2}{2N} \quad (10)$$

Starting from a random initial guess Θ_0 , the standard (naive) gradient descent would update the network by changing its parameters in the directing that most quickly reduces the loss (equation 10):

$$\Delta\Theta = -\alpha \nabla_{\Theta} \mathcal{L} \quad (11)$$

Stochastic gradient descent approximates the gradient of the loss function by the average over a small subsample of the data set $\{\beta_i, Y_i\}_{i=1}^N$:

$$\Delta\Theta = -\frac{\alpha}{n} \sum_{i=1}^n (Y_i - \hat{g}(\beta_i; \Theta_0)) \nabla_{\Theta} \hat{g}$$

and iterates until the loss function flattens. Once training is complete, the network $\hat{g}(\beta)$ learned a differentiable mapping between parameters and the moments of interest. This has two major implications for structural estimation. It is a fast way to evaluate moments for different sets of parameters. In the application of section 4, for instance, it takes less than a millisecond to evaluate all 12 moments for a given set of parameters. In

⁸We can generate an arbitrarily large data set using the policy functions $C(\mathbf{X}, \beta)$

addition, we can explore the structure of this mapping to search for local optima in the outer loop. Specifically, we have analytical gradients and Hessians that can be explored by a root finding Newton method.

4 A DSGE model with irreversible investment

In this section I illustrate the approximate policy iteration and the moment learning algorithms with a neoclassical growth model with stochastic volatility, endogenous labor supply and irreversible investment. The experiment is as follows. Given a fixed set of model parameters, I solve the model and generate artificial data that will be used in the estimation.⁹ The estimation task is to recover the deep structural parameters of the model by observing this data.

The first step is to solve the model for an entire range of the parameters. Once we learn the policy networks, we can simulate different paths for different parameter configurations and store observations in a data set. Next, the moment network is trained on this data set to learn the mapping between parameters and moments. Finally, an optimizer explores this mapping to find the set of parameters that minimizes that a distance between the model-implied moments and the artificial moments.

The model is similar to the setup in [Aruoba et al. \(2006\)](#) and [Caldara et al. \(2012\)](#), which compares different solution methods used in economics. This model is chosen for the three reasons. First, the setup is simple and familiar to most economists. Second, since labor supply is endogenous, we cannot write the optimal control as an explicit function of the value function, which is a pre-requisite to use the algorithm in [Duarte \(2018\)](#). Third, irreversibility of investment implies that the first-order conditions are

⁹I used both Dynare and a small network with fixed model parameters to generate the data, and both approaches yield the same moments.

valid only as inequalities. In that case, perturbation methods (Judd and Guu (1997)) produce non-trivial errors for a sufficiently high capital stock.

The model setup is as follows. The central planner chooses aggregate consumption C and labor L to maximize a expected lifetime utility:

$$\max_{C,L} \mathbb{E} \int_0^{\infty} e^{-\beta t} \frac{(C_t^{\nu} L_t^{1-\nu})^{1-\gamma}}{1-\gamma} dt \quad (12)$$

The final good is produced with a Cobb-Douglas technology $Y = e^z K^{\zeta} L^{1-\zeta}$, where K is the capital stock and z is the productivity, that follows:

$$dz = -(1-\lambda)zdt + e^{\sigma} dZ_1 \quad (13)$$

The productivity volatility is itself stochastic and follows:

$$d\sigma = (1-\rho)(\bar{\sigma} - \sigma)dt + \eta dZ_2 \quad (14)$$

where the Brownian shocks dZ_1 and dZ_2 are independent.

Capital is subject to a depreciation rate δ , and accumulates according to:

$$dK = (I - \delta K)dt \quad (15)$$

where $I \equiv Y - C$.

The central planner value function is therefore:

$$\begin{aligned}
V(K, z, \sigma) &= \max_{C, L} \mathbb{E} \left[\int_0^\infty \frac{(C_t^\nu L_t^{1-\nu})^{1-\gamma}}{1-\gamma} dt \right] \\
&\quad s.t. \\
dK &= (e^z K^\zeta L^{1-\zeta} - C - \delta K) dt \\
dz &= -(1 - \lambda) z dt + e^\sigma dZ_1 \\
d\sigma &= (1 - \rho)(\bar{\sigma} - \sigma) dt + \eta dZ_2
\end{aligned} \tag{16}$$

And the policies C, L are defined implicitly by:

$$\begin{aligned}
C, L &= \arg \max_{c, l} HJB(K, z, \sigma, c, l) \\
HJB(K, z, \sigma, c, l) &\equiv u(c, l) - \rho V + \frac{\mathbb{E} dV}{dt}(K, z, \sigma, c, l)
\end{aligned} \tag{17}$$

Since labor L is restricted to the interval $[0, 1]$, the labor network is parametrized by $L(\cdot) = \sigma(DNN(\cdot))$, where σ is the logistic function and DNN is a generic neural network. Likewise, since investment is irreversible, the consumption network is parametrized by $C(\cdot) = Y \cdot \sigma(DNN(\cdot))$. The architecture of the value and policy networks is shown in table 1.

Figure 1 shows the numerical solutions for the value function when the productivity z and the stochastic volatility σ are held constant at their steady states, and the parameters fixed at some reference values. The solid blue line is the result of the policy iteration algorithm of section 2, the dashed red line is the solution from a second-order perturbation, and the dashed green line is corresponds to a third-order perturbation. The dashed gray

line indicates the capital steady state. It is well understood that perturbation methods produce good approximations near that steady state of neoclassical growth models, so it is a good check to verify that the policy iteration result coincides with the results from standard perturbation. Away from the steady state, however, perturbation methods may produce very inaccurate results, as figure 1 shows. In particular, for a large enough capital stock, the value function corresponding to a second order perturbation is decreasing in capital, while a third order perturbation produces a non-concave value function.

Figure 2 shows the optimal policies, and illustrate the main drawback of local methods. Near the steady state, the first-order conditions hold with equality, and perturbation produces good results. Away from the steady state, the central planner hits a constraint. For sufficiently high capital, he would like to disinvest and increase consumption. If capital is illiquid, therefore, the Euler equation for consumption holds with strict inequality. The agent's behavior in that region is completely different from his behavior near the steady state and therefore local methods fail spectacularly. Global methods, like the policy iteration of section 2, don't suffer from this problem and can capture different behaviors in different regions of the state space, as illustrated in panel (b) of figure 2. As an additional, check, notice that for a sufficiently high level of capital, we can compute analytically the optimal level of labor supply L . Indeed, for high K , the representative agent consumes the entire output Y . Therefore the optimal labor supply satisfies $L = \arg \max L^{(1-\zeta)\nu}(1-L)^{1-\nu} \approx 0.285$, which is almost exactly the value found by the policy network.

The next step is to generate observations of variables of interest using the learned policies, and train the moments network on this data set. Table 2 shows the targeted moments, their true values and the learned moments, and indicates that moments network provides an excellent fit even with a small architecture (table 3), small training set (1 M observations) and no hyperparameters tuning.

Finally, an optimizer tries to find the global minimum of:

$$d(\boldsymbol{\beta}) \equiv (\hat{g} - g(\boldsymbol{\beta}))' W (\hat{g} - g(\boldsymbol{\beta})) \quad (18)$$

where $\hat{g}$ are the data moments, $g(\boldsymbol{\beta})$ is the output of the moments network and W is a positive definite weighting matrix. In this exercise I use $W = \text{diag}(1/\hat{g}^2)$ to remove scaling effects. The use of a global optimization routine is important because the distance function d may have many local optima. In this exercise I use the Basin-Hopping algorithm (Wales and Doye (1997)), which at each iteration performs: 1) random perturbation of the parameters; 2) a gradient-based local minimization; and 3) accept / reject the new parameters according to a Metropolis criterion.

Table 4 shows main result of this exercise, which is the output of the estimation. The basin-hopping minimization routine took less than a minute to find the candidate of a global minimum, with a loss of $d = 10^{-4}$. As table 4 makes it clear, the estimated parameters are remarkably close to the true parameters that were used to generate the artificial data.

5 Conclusion

I show that gradient-based optimization methods can be used to efficiently estimate stochastic dynamic economic models by constructing the binding-function directly from simulated observations prior to the optimization loop. After a *moments network* is trained on that data set, optimization can be carried out as if we had the estimating equations in closed form. In particular, we can take advantage of the gradient and hessian of the loss function to search for the local minimum. Essentially, Simulated Methods of Moments (SMM) becomes as tractable as the Generalized Methods of Moments (GMM).

The paper dispels two widely held beliefs about simulated moments structural estimation. One of these beliefs is that it is infeasible to use gradient-based methods to find the parameters that minimize the corresponding loss function.¹⁰ In fact, gradients are used in four parts of the algorithm: in the policy evaluation step, in the policy improvement step, in learning the moments network, and in the outer loop minimization step. I also show that structural estimation does not have to be difficult. The solution and estimation of the benchmark model were done without writing down a single equation (with the exception of the model setup), relying on general-purpose libraries to perform mechanic operations like the application of Ito's lemma for multivariate functions, Bellman updates and the computation of policy gradients. The solution and estimation of a macroeconomic model with three state variables and eight parameters took approximately three hours on a personal computer.

¹⁰See [Strebulaev and Whited \(2012\)](#), for example.

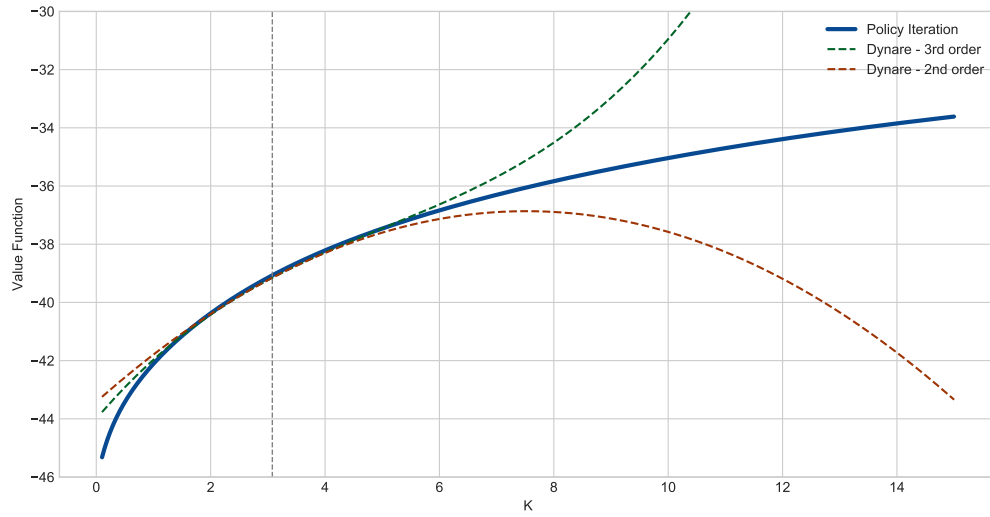
References

- Achdou, Y., Buera, F. J., Lasry, J.-M., Lions, P.-L., Moll, B., 2014. Partial differential equation models in macroeconomics. *Philosophical Transactions of the Royal Society of London A: Mathematical, Physical and Engineering Sciences* 372.
- Aruoba, S. B., Fernandez-Villaverde, J., Rubio-Ramirez, J. F., 2006. Comparing solution methods for dynamic equilibrium economies. *Journal of Economic Dynamics and Control* 30, 2477–2508.
- Berry, S., Levinsohn, J., Pakes, A., 1995. Automobile prices in market equilibrium. *Econometrica* 63, 841–90.
- Bertsekas, D. P., 2017. *Dynamic Programming and Optimal Control*. Athena Scientific, fourth ed.
- Brunnermeier, M., Sannikov, Y., 2016. Macro, money, and finance: A continuous-time approach. Elsevier, vol. 2 of *Handbook of Macroeconomics*, pp. 1497 – 1545.
- Caldara, D., Fernández-Villaverde, J., Rubio-Ramírez, J. F., Yao, W., 2012. Computing {DSGE} models with recursive preferences and stochastic volatility. *Review of Economic Dynamics* 15, 188 – 206.
- Duarte, V., 2018. Machine learning for continuous-time economics. SSRN .
- Duffie, D., Singleton, K. J., 1993. Simulated moments estimation of markov models of asset prices. *Econometrica* 61, 929–952.
- Durham, G. B., 2006. Monte carlo methods for estimating, smoothing, and filtering one- and two-factor stochastic volatility models. *Journal of Econometrics* 133, 273 – 305.
- Goodfellow, I., Bengio, Y., Courville, A., 2016. *Deep learning*. MIT press.
- Gourieroux, C., Monfort, A., Renault, E., 1993. Indirect inference. *Journal of Applied Econometrics* 8, S85–S118.
- Hansen, L. P., 1982. Large sample properties of generalized method of moments estimators. *Econometrica* 50, 1029–1054.
- Hennessy, C. A., Whited, T. M., 2005. Debt dynamics. *The Journal of Finance* 60, 1129–1165.
- Judd, K. L., Guu, S.-M., 1997. Asymptotic methods for aggregate growth models. *Journal of Economic Dynamics and Control* 21, 1025–1042.
- Lee, B.-S., Ingram, B., 1991. Simulation estimation of time-series models. *Journal of Econometrics* 47, 197–205.
- McFadden, D., 1989. A method of simulated moments for estimation of discrete response models without numerical integration. *Econometrica* 57, 995–1026.

- Norets, A., 2012. Estimation of dynamic discrete choice models using artificial neural network approximations. *Econometric Reviews* 31, 84–106.
- Pakes, A., Pollard, D., 1989. Simulation and the asymptotics of optimization estimators. *Econometrica* 57, 1027–1057.
- Scheidegger, S., Billionis, I., 2017. Machine learning for high-dimensional dynamic stochastic economies. SSRN .
- Smith, A. A., 2005. Estimating nonlinear timeseries models using simulated vector autoregressions. *Journal of Applied Econometrics* 8, S63–S84.
- Strebulaev, I. A., Whited, T., 2012. Dynamic models and structural estimation in corporate finance. *Foundations and Trends(R) in Finance* 6, 1–163.
- Sutton, R. S., 1988. Learning to predict by the methods of temporal differences. *Machine Learning* 3, 9–44.
- Szepesvari, C., 2010. *Algorithms for Reinforcement Learning*. Morgan and Claypool Publishers.
- Wales, D. J., Doye, J. P. K., 1997. Global optimization by basin-hopping and the lowest energy structures of lennard-jones clusters containing up to 110 atoms. *The Journal of Physical Chemistry A* 101, 5111–5116.

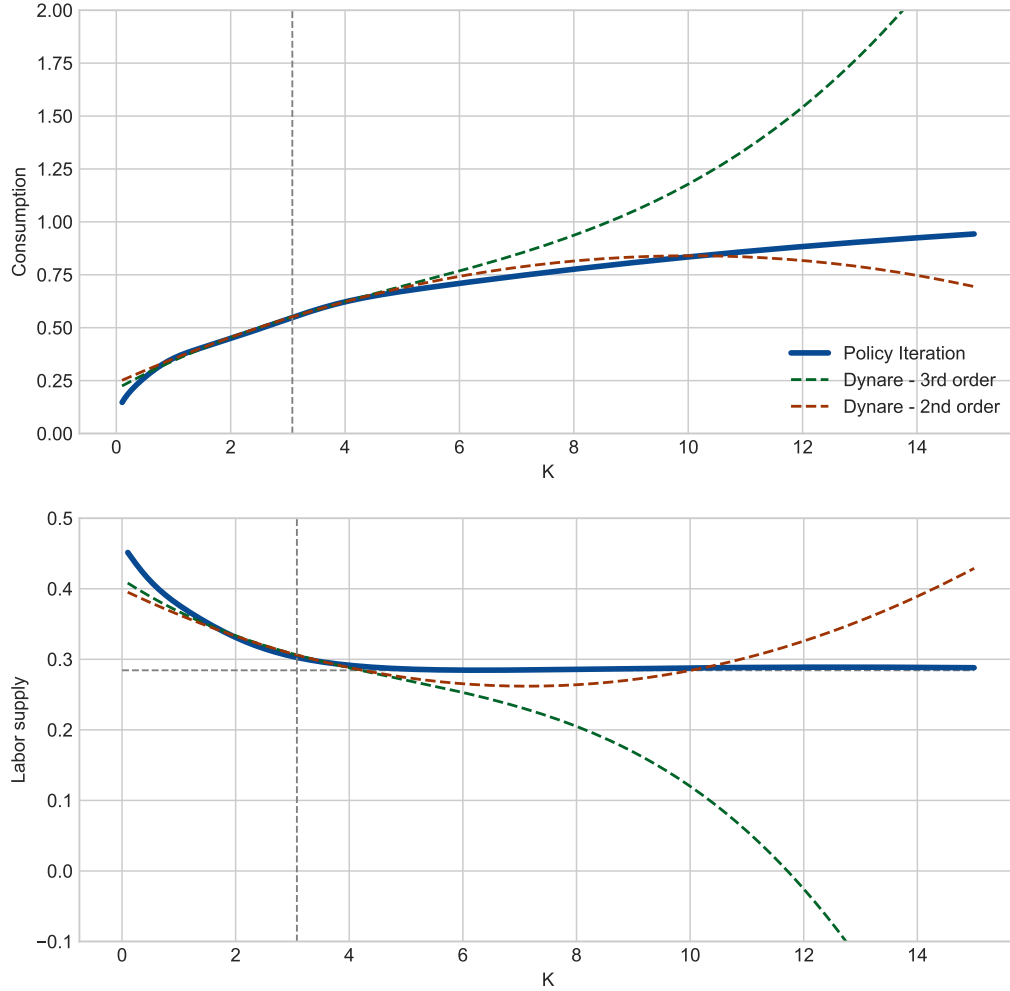
A Figures

Figure 1: Cross section of the value function



This figure shows the value function for $z = 0$, $\sigma = \bar{\sigma}$, $\gamma = 2$, $\beta = 0.04$, $\nu = 0.36, 0.30$, $\lambda = 0.95, -3.86$, $\eta = 0.1$ and $\rho = 0.9$. The blue line is the output of the policy iteration algorithm with neural networks. The red and green dashed lines are the solutions obtained with perturbation methods, plotted here for comparison. The vertical gray dashed line corresponds to the capital steady state.

Figure 2: Cross section of the policy functions



This figure shows the policy functions C and L for $z = 0$, $\sigma = \bar{\sigma}$, $\gamma = 2$, $\beta = 0.04$, $\nu = 0.36, 0.30$, $\lambda = 0.95, -3.86$, $\eta = 0.1$ and $\rho = 0.9$. The blue line is the output of the policy iteration algorithm with neural networks. The red and green dashed lines are the solutions obtained with perturbation methods, plotted here for comparison. The vertical gray dashed line corresponds to the capital steady state. The horizontal gray line corresponds to the analytical value for L when $K \rightarrow \infty$.

B Tables

Table 1: Value and Policy Networks Architecture

Numer of hidden layers	2
Hidden units	[100, 100]
Activation Function	tanh
Type gradient descent	RMSProp
Learning rate	1e-5
Momentum	.99
Mini batch size	1000

Table 2: Fitted binding function

	True moment	Fitted moment
K	3.11	3.28
σ	-3.86	-3.79
C	0.55	0.55
L	0.30	0.30
Y	0.61	0.61
r_f	0.04	0.04
σ^2	14.97	14.89
C^2	0.31	0.32
L^2	0.092	0.094
$\sigma_t \cdot \sigma_{t-1}$	14.97	14.89
$\sigma_t^2 \cdot \sigma_{t-1}$	-58.23	-57.56

Table 3: Moments Network

<i>Hyperparameters</i>	
Numer of hidden layers	3
Hidden units	[150, 50]
Activation Function	<i>ReLU</i>
Type gradient descent	NAdam
Learning rate	1e-5
Mini batch size	50

Table 4: Parameters estimates

	Parameter	True Value	Estimated value
<i>Model parameters</i>			
Coefficient relative risk aversion	γ	2.00	1.81
Time discount	β	0.04	Fixed
Elasticity of labor supply	ν	0.36	0.36
Capital share	ζ	0.30	0.31
Productivity autocorrelation	λ	0.95	0.93
Mean volatility	$\bar{\sigma}$	-3.86	-3.88
Volatility of the volatility	η	0.1	0.1
Volatility autocorrelation.	ρ	0.9	0.95

C Derivations

C.1 Policy improvement

Starting with a network policy $C(\mathbf{X}; \Theta_0)$ and the associated value function V^{C_0} , let

$$HJB(\mathbf{X}, V^{C_0}, C_1) = U(C_1) + \frac{\mathbb{E}dV^{C_0}}{dt}(\mathbf{X}, \mathbf{C}_1)$$

Assuming that $HJB(V^{C_0}, \mathbf{X}, C_1)$ is convex in C_1 , the policy $C_0 + \nabla_{C_1} HJB$ is an improvement over the policy C_0 . We can use these values as targets to adjust the policy network with stochastic gradient descent:

$$\begin{aligned}\Theta_C &= -\frac{\alpha}{n} \sum_{i=1}^n \nabla_C HJB_i \nabla_{\Theta_C} C_i \\ &= -\frac{\alpha}{n} \sum_{i=1}^n \nabla_{\Theta_C} HJB_i\end{aligned}$$

where the first line is the standard SGD update and the second line is a direct application of the chain rule.

C.2 Conditional expectation minimizes the mean squared errors

This derivation can be found in any econometrics textbook. It is reproduced here for convenience.

$$\begin{aligned}(Y - f(\beta))^2 &= (Y - \mathbb{E}[Y|\beta] + \mathbb{E}[Y|\beta] - f(\beta))^2 \\&= (Y - \mathbb{E}[Y|\beta])^2 + (\mathbb{E}[Y|\beta] - f(\beta))^2 + 2(Y - \mathbb{E}[Y|\beta])(\mathbb{E}[Y|\beta] - f(\beta)) \\&\Rightarrow\end{aligned}$$

$$\mathbb{E}[(Y - f(\beta))^2] = \mathbb{E}[(Y - \mathbb{E}[Y|\beta])^2] + \mathbb{E}[(\mathbb{E}[Y|\beta] - f(\beta))^2]$$

The first term in the right-hand side of the last equation doesn't depend on f , so the expression is clearly minimized when $f(\beta) = \mathbb{E}[Y|\beta]$.